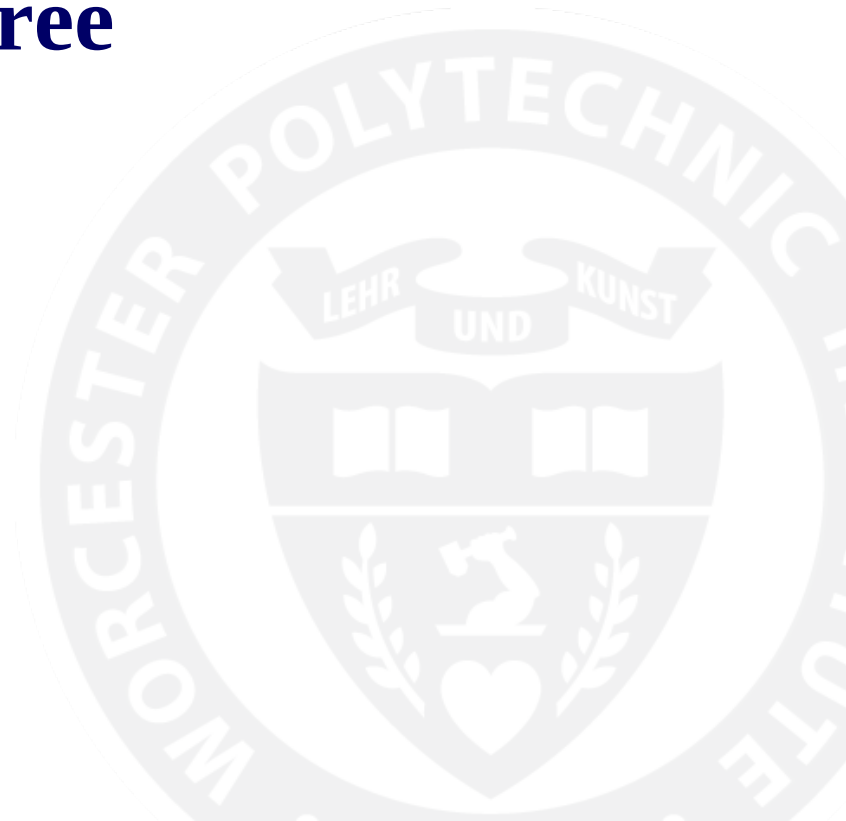




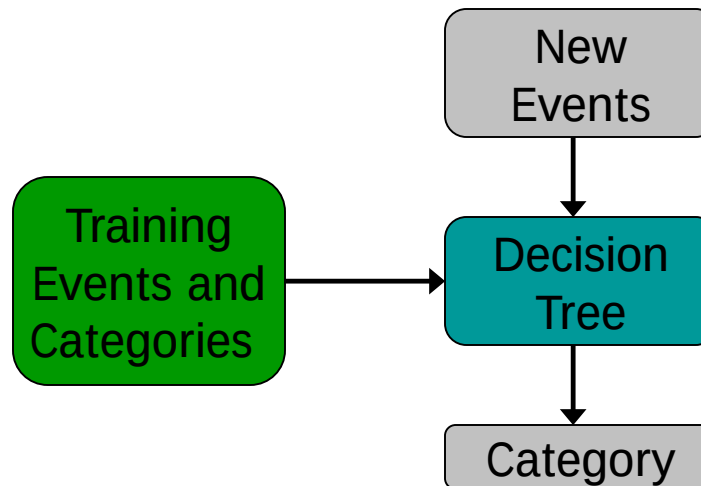
WPI

Lecture 3a: Decision Tree



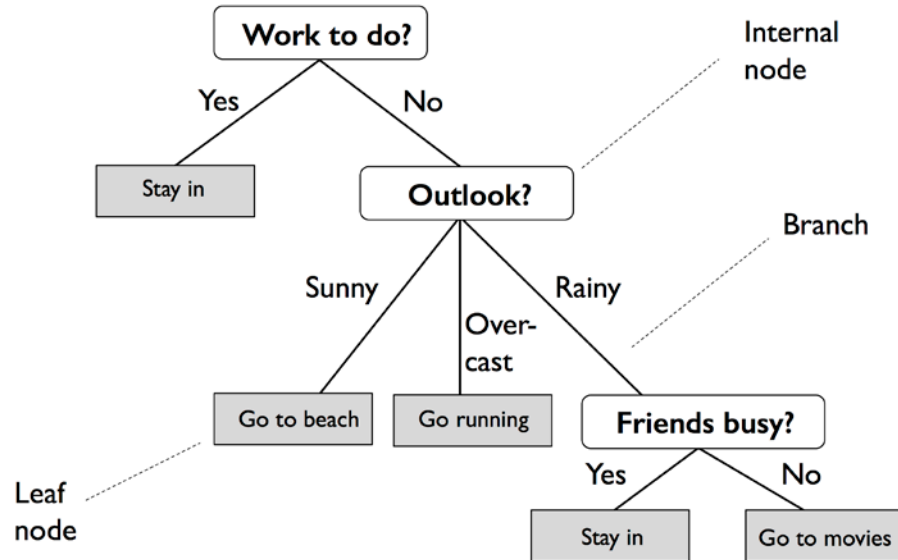
Introduction

- Use a decision tree to predict categories for new events.
- Use training data to build the decision tree.

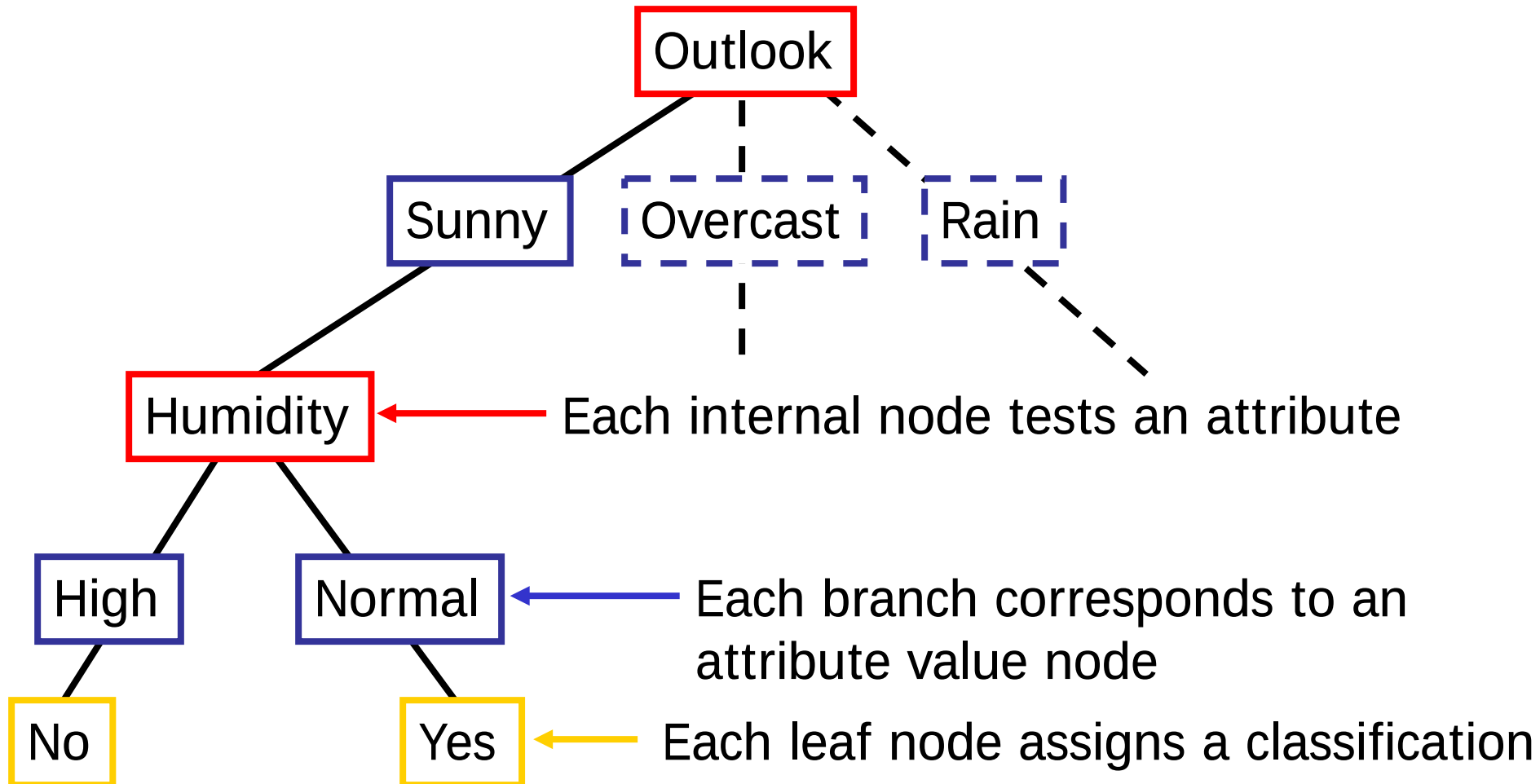


Decision Tree

Linear regression is too simple to handle complex problems. So, let's look at another more powerful, explainable and also intuitive method, decision tree. Decision tree mainly works on discrete variables, but it also can work on continuously variables by discretizing them.



Decision Tree for Playing Tennis



Decision Tree

- The basic idea behind any decision tree algorithm is as follows:
 - Choose the **best** feature(s) to split the remaining instances and make that feature a decision node
 - Repeat this process recursively for each child
 - Stop when:
 - All the instances have the same target feature value
 - There are no more features
 - There are no more instances

Decision Tree

In order to split the nodes at the most informative features, we need to define an objective function that we want to optimize via the tree learning algorithm. Here, our objective function is to maximize the information gain at each split, which we define as follows:

$$IG(D_p, f) = I(D_p) - \sum_{j=1}^m \frac{N_j}{N_p} I(D_j)$$

Here, f is the feature to perform the split, D_p and D_j are the dataset of the parent and j th child node, I is our **impurity** measure, N_p is the total number of samples at the parent node, and N_j is the number of samples in the j th child node. As we can see, the information gain is simply the difference between the impurity of the parent node and the sum of the child node impurities — the lower the impurity of the child nodes, the larger the information gain. However, for simplicity and to reduce the combinatorial search space, most libraries (including scikit-learn) implement binary decision trees. This means that each parent node is split into two child nodes, D_{left} and D_{right} :

$$IG(D_p, f) = I(D_p) - \frac{N_{left}}{N_p} I(D_{left}) - \frac{N_{right}}{N_p} I(D_{right})$$

Decision Tree

How to measure **impurity**?

Gini impurity (I_G):

$$I_G(t) = \sum_{i=1}^c p(i|t)(1 - p(i|t)) = 1 - \sum_{i=1}^c p(i|t)^2$$

Entropy (I_H):

$$I_H(t) = - \sum_{i=1}^c p(i|t) \log_2 p(i|t)$$

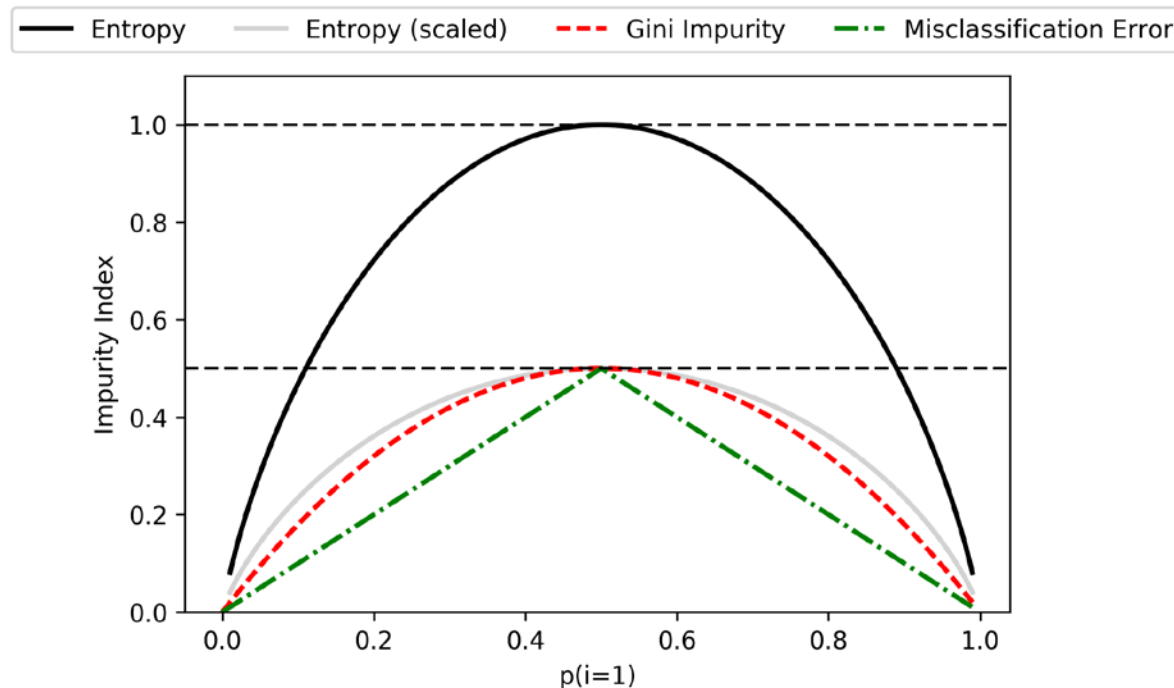
$p(i|t)$ represents the percentage of samples in node t belongs to class i .

Decision Tree

Another impurity measure is the classification error:

$$I_E = 1 - \max \{p(i|t)\}$$

This is a useful criterion for pruning but not recommended for growing a decision tree, since it is less sensitive to changes in the class probabilities of the nodes.



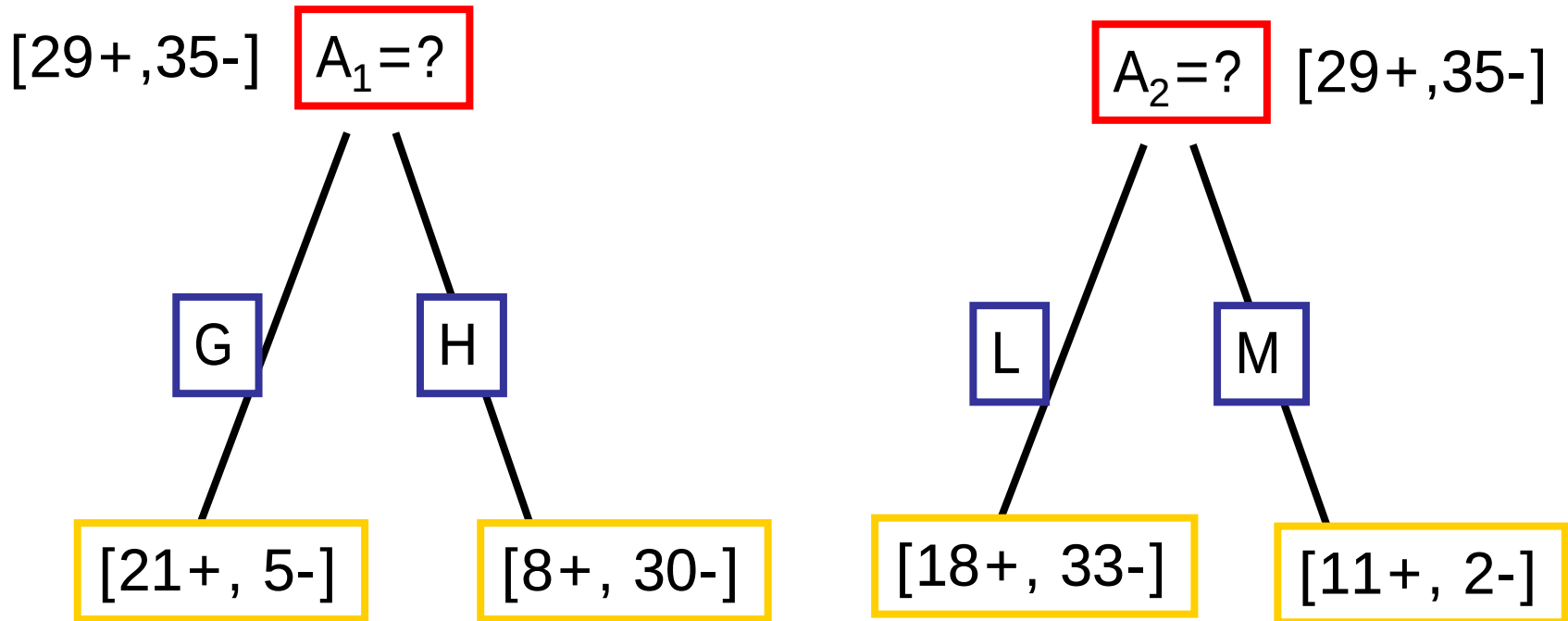
When to consider Decision Trees

- Instances describable by attribute-value pairs
- Target function is discrete valued
- Disjunctive hypothesis may be required
- Possibly noisy training data
- Missing attribute values
- Examples:
 - Medical diagnosis
 - Credit risk analysis
 - Object classification for robot manipulator (Tan 1993)

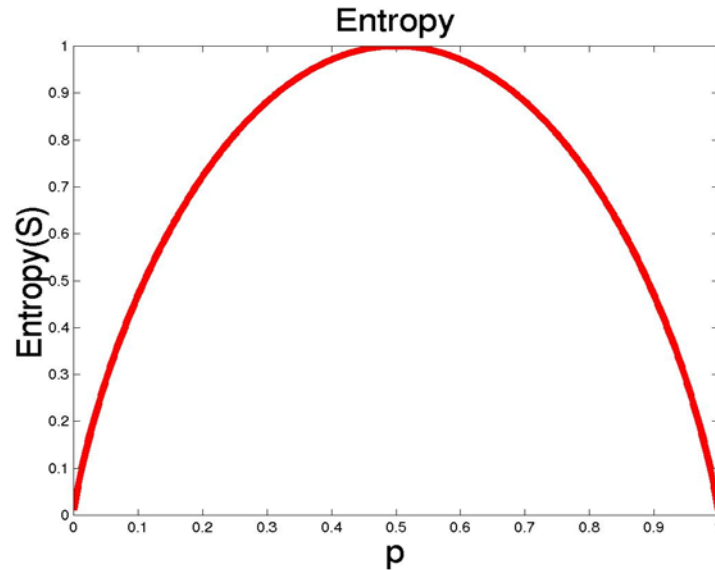
Top-Down Induction of Decision Trees ID3

1. $A \leftarrow$ the “best” decision attribute for next *node*
2. Assign A as decision attribute for *node*
3. For each value of A create new descendant
4. Sort training examples to leaf node according to the attribute value of the branch
5. If all training examples are perfectly classified (same value of target attribute) stop, else iterate over new leaf nodes.

Which attribute is best?



Entropy



- S is a sample of training examples
- p_+ is the proportion of positive examples
- p_- is the proportion of negative examples
- Entropy measures the impurity of S

$$\text{Entropy}(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

Entropy

- Entropy(S) = expected number of bits needed to encode class (+ or -) of randomly drawn members of S (under the optimal, shortest length-code)

Why?

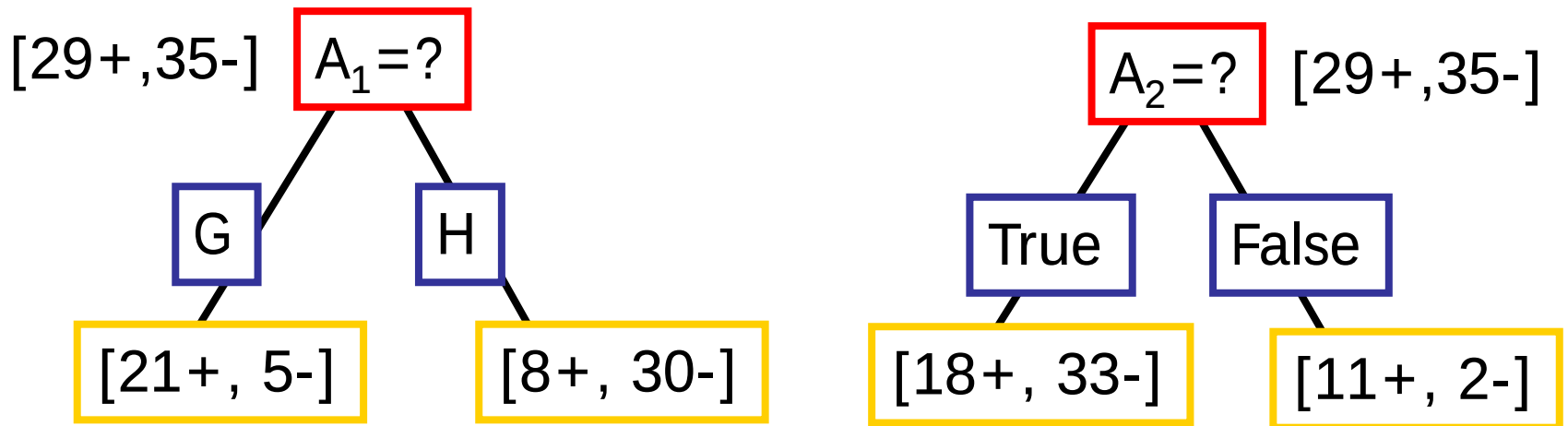
- Information theory optimal length code assign $-\log_2 p$ bits to messages having probability p .
- So the expected number of bits to encode (+ or -) of random member of S:
$$-p_+ \log_2 p_+ - p_- \log_2 p_-$$

Information Gain (S=E)

- Gain(S,A): expected reduction in entropy due to sorting S on attribute A

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in D_A} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$\begin{aligned} Entropy([29+, 35-]) &= -29/64 \log_2 29/64 - 35/64 \log_2 35/64 \\ &= 0.99 \end{aligned}$$



Information Gain

$$\text{Entropy}([21+, 5-]) = 0.71$$

$$\text{Entropy}([8+, 30-]) = 0.74$$

$$\text{Gain}(S, A_1) = \text{Entropy}(S)$$

$$-26/64 * \text{Entropy}([21+, 5-])$$

$$-38/64 * \text{Entropy}([8+, 30-])$$

$$= 0.27$$

$$\text{Entropy}([18+, 33-]) = 0.94$$

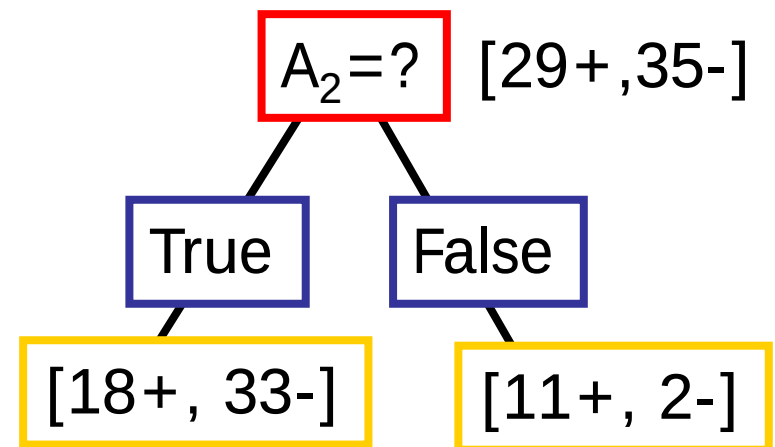
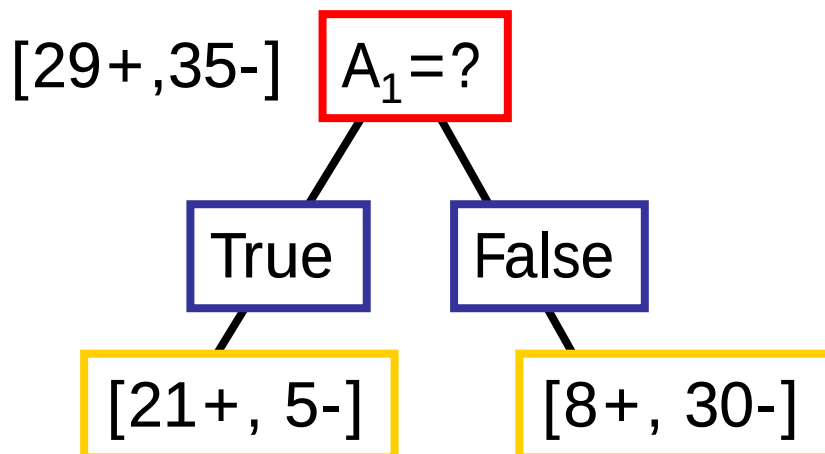
$$\text{Entropy}([11+, 2-]) = 0.62$$

$$\text{Gain}(S, A_2) = \text{Entropy}(S)$$

$$-51/64 * \text{Entropy}([18+, 33-])$$

$$-13/64 * \text{Entropy}([11+, 2-])$$

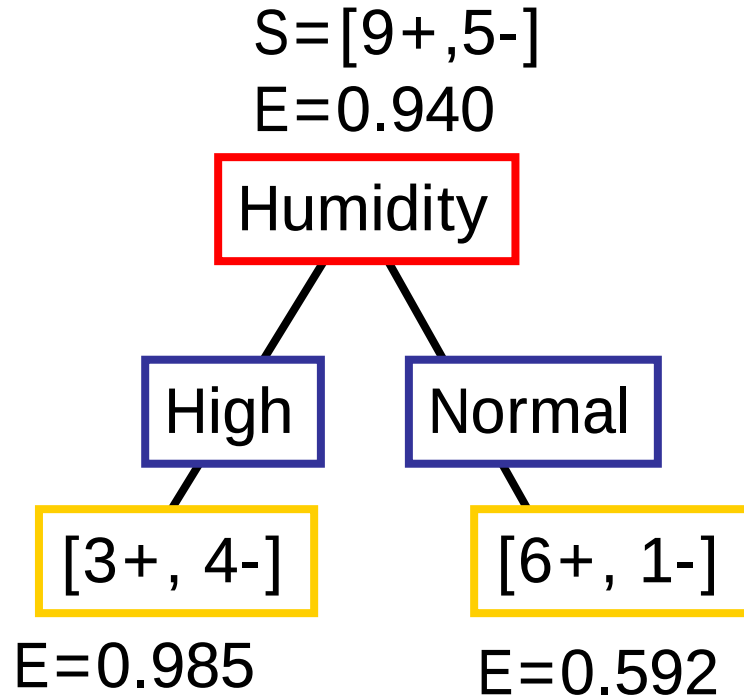
$$= 0.12$$



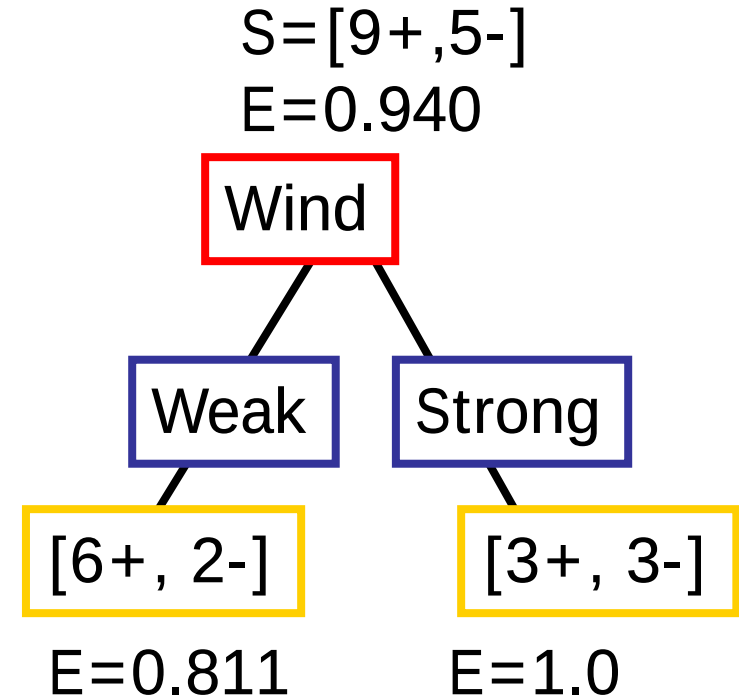
Training Examples

Day	Outlook	Temp.	Humidity	Wind	Play Tennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Weak	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cold	Normal	Weak	Yes
D10	Rain	Mild	Normal	Strong	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Selecting the Next Attribute



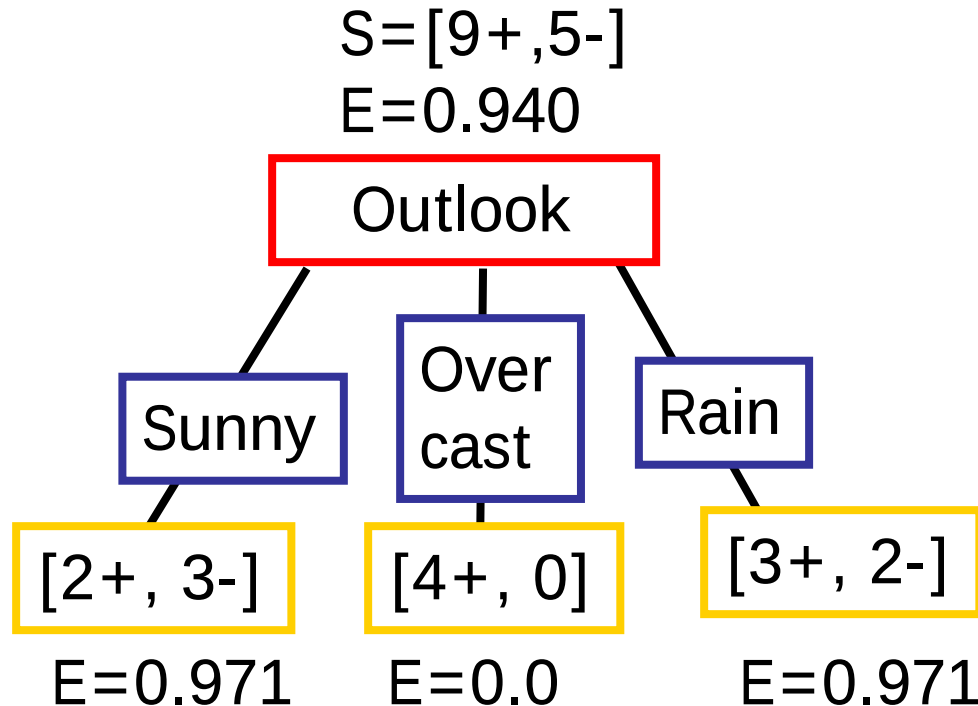
$$\begin{aligned}\text{Gain}(S, \text{Humidity}) &= 0.940 - (7/14) * 0.985 \\ &\quad - (7/14) * 0.592 \\ &= 0.151\end{aligned}$$



$$\begin{aligned}\text{Gain}(S, \text{Wind}) &= 0.940 - (8/14) * 0.811 \\ &\quad - (6/14) * 1.0 \\ &= 0.048\end{aligned}$$

Humidity provides greater info. gain than Wind, w.r.t target classification. 17

Selecting the Next Attribute



$$\begin{aligned} \text{Gain}(S, \text{Outlook}) &= 0.940 - (5/14) * 0.971 \\ &\quad - (4/14) * 0.0 - (5/14) * 0.0971 \\ &= 0.247 \end{aligned}$$

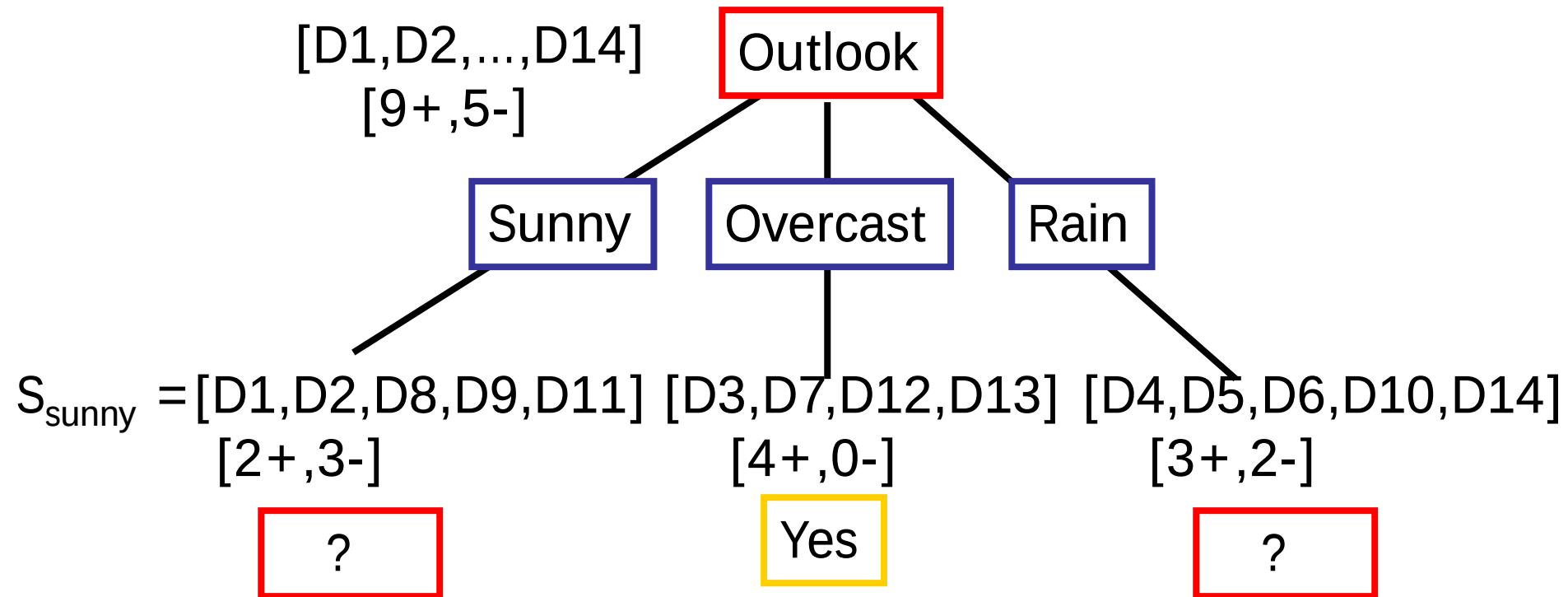
Selecting the Next Attribute

The information gain values for the 4 attributes are:

- $\text{Gain}(S, \text{Outlook}) = 0.247$
- $\text{Gain}(S, \text{Humidity}) = 0.151$
- $\text{Gain}(S, \text{Wind}) = 0.048$
- $\text{Gain}(S, \text{Temperature}) = 0.029$

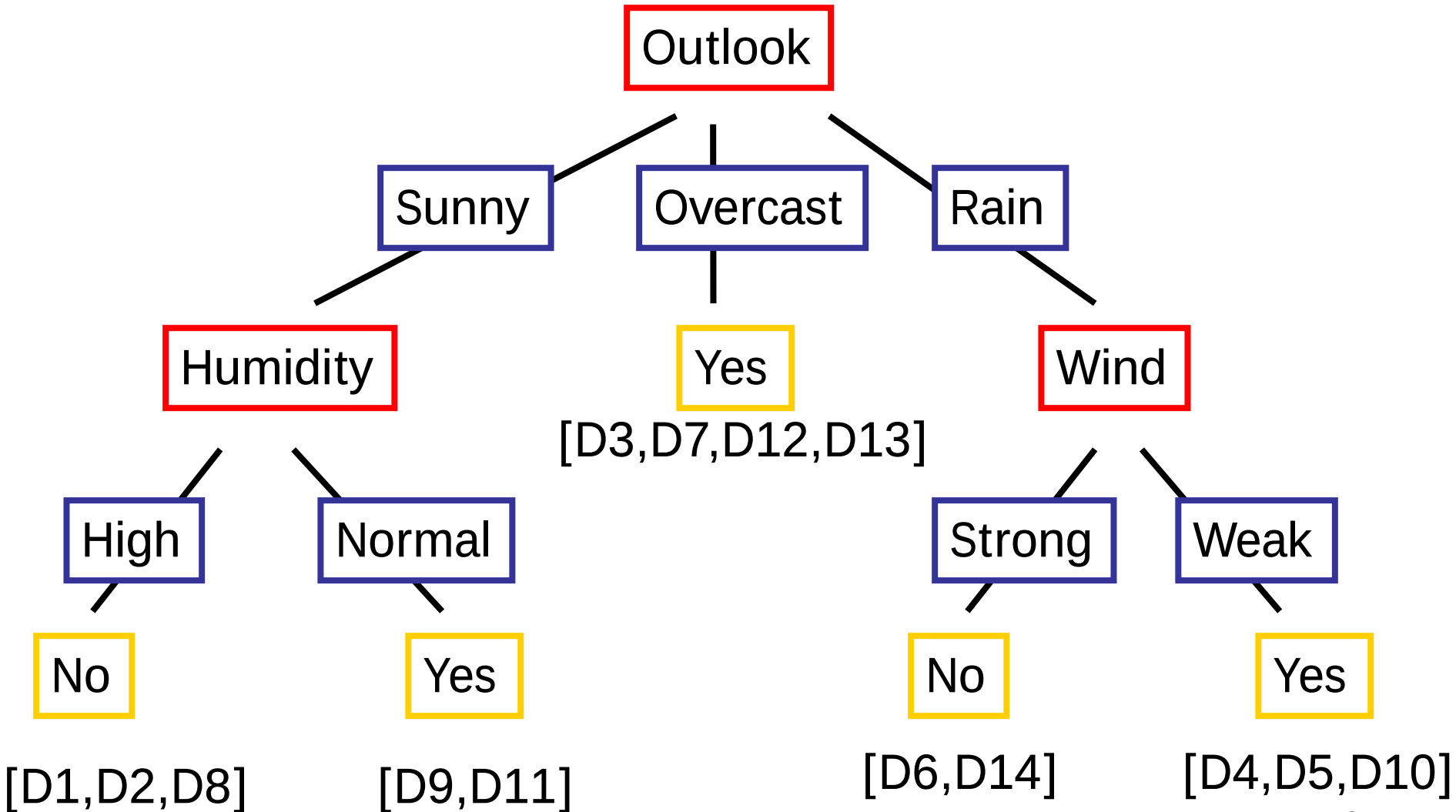
where S denotes the collection of training examples

ID3 Algorithm

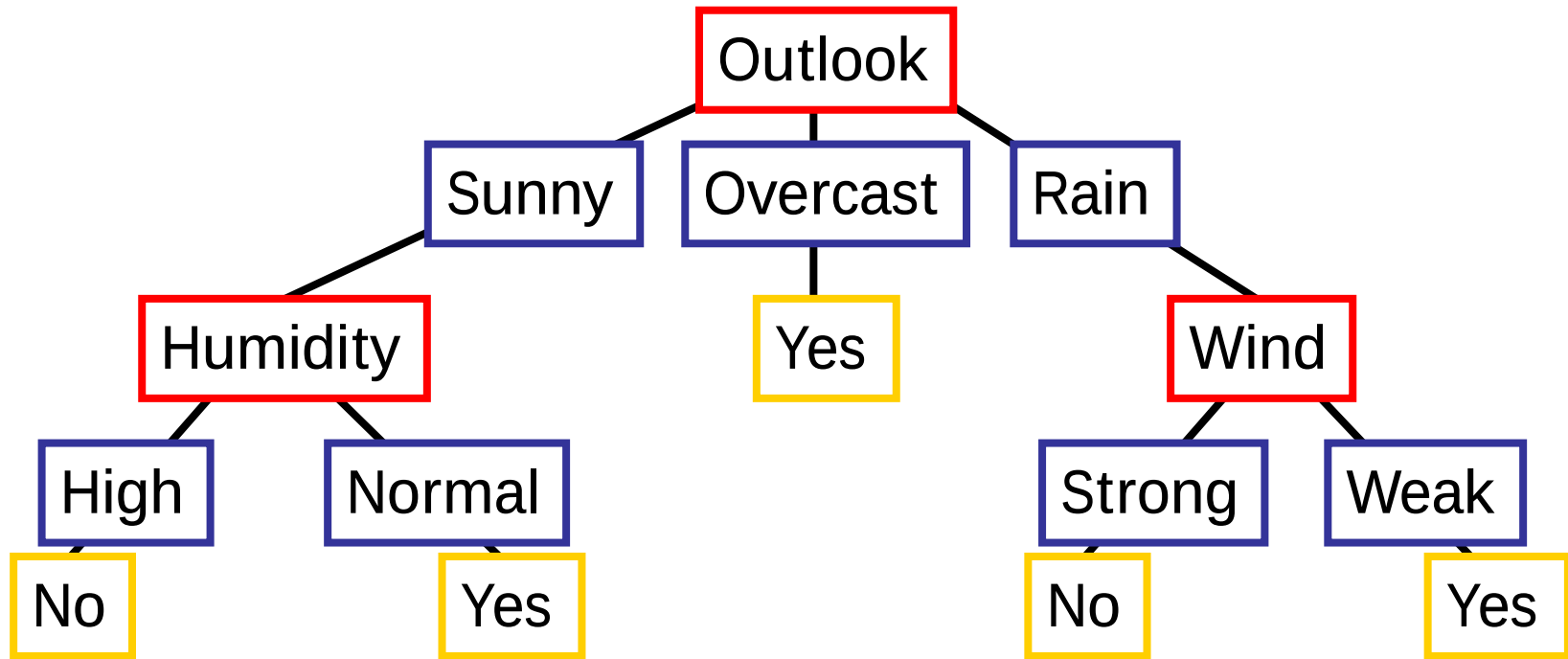


$$\begin{aligned} \text{Gain}(S_{\text{sunny}}, \text{Humidity}) &= 0.970 - (3/5)0.0 - 2/5(0.0) = 0.970 \\ \text{Gain}(S_{\text{sunny}}, \text{Temp.}) &= 0.970 - (2/5)0.0 - 2/5(1.0) - (1/5)0.0 = 0.570 \\ \text{Gain}(S_{\text{sunny}}, \text{Wind}) &= 0.970 - (2/5)1.0 - 3/5(0.918) = 0.019 \end{aligned}$$

ID3 Algorithm



Converting a Tree to Rules



- R_1 : If (Outlook=Sunny) $\wedge$ (Humidity=High) Then PlayTennis=No
 R_2 : If (Outlook=Sunny) $\wedge$ (Humidity=Normal) Then PlayTennis=Yes
 R_3 : If (Outlook=Overcast) Then PlayTennis=Yes
 R_4 : If (Outlook=Rain) $\wedge$ (Wind=Strong) Then PlayTennis=No
 R_5 : If (Outlook=Rain) $\wedge$ (Wind=Weak) Then PlayTennis=Yes

Occam's Razor

"If two theories explain the facts equally well, then the simpler theory is to be preferred"

Arguments in favor:

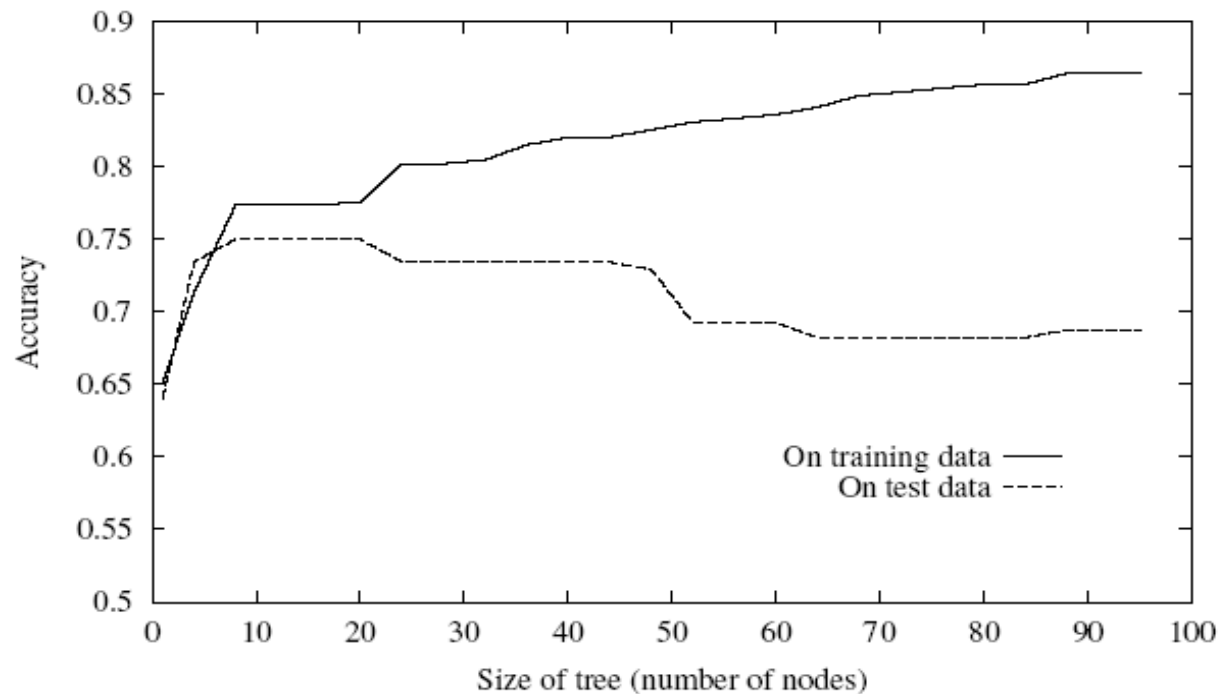
- Fewer short hypotheses than long hypotheses
- A short hypothesis that fits the data is unlikely to be a coincidence
- A long hypothesis that fits the data might be a coincidence

Arguments opposed:

- There are many ways to define small sets of hypotheses

Overfitting

- One of the biggest problems with decision trees is **Overfitting**



Avoid Overfitting

- stop growing when split not statistically significant
- grow full tree, then post-prune

Select “best” tree:

- measure performance over training data
- measure performance over separate validation data set
- $\min(|\text{tree}| + |\text{misclassifications}(\text{tree})|)$

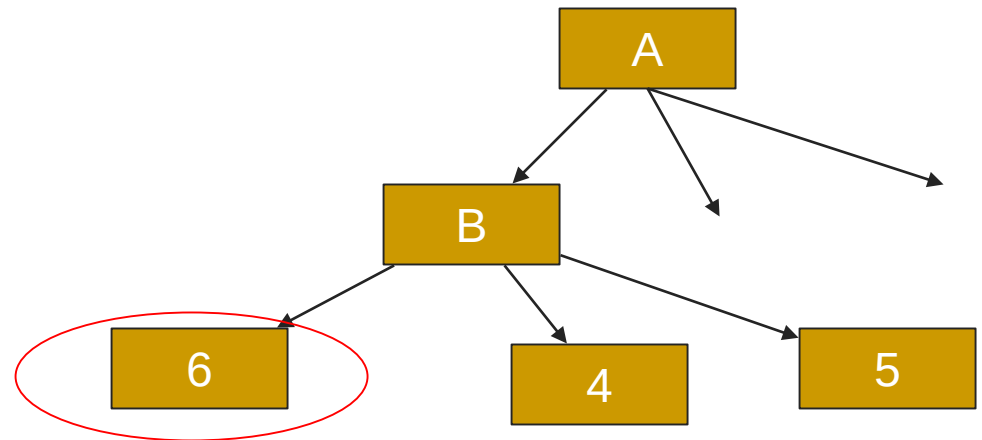
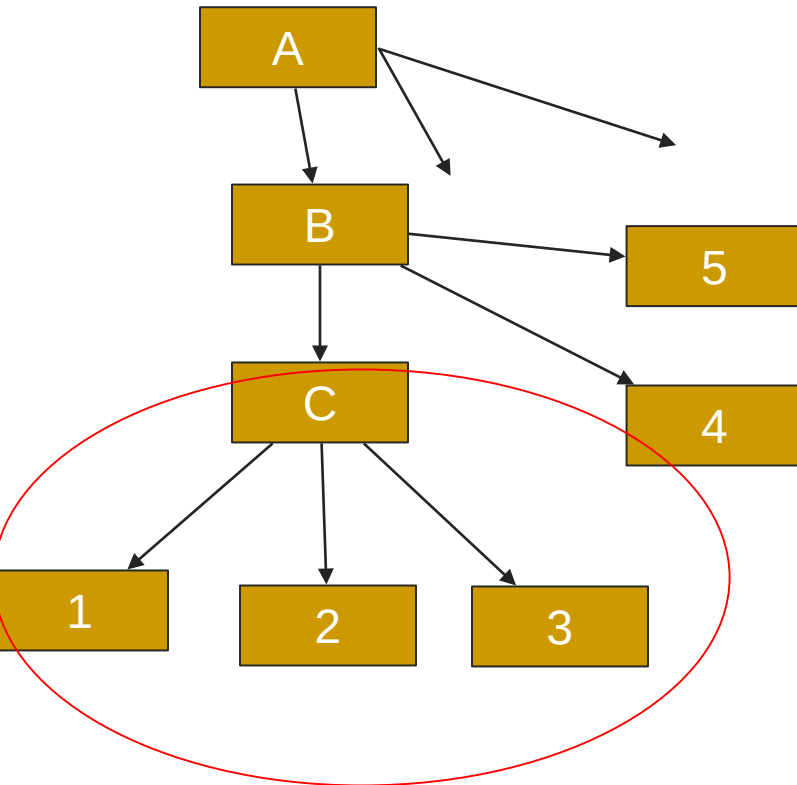
Decision Tree

If the tree is going to be too complex, we can prune it.

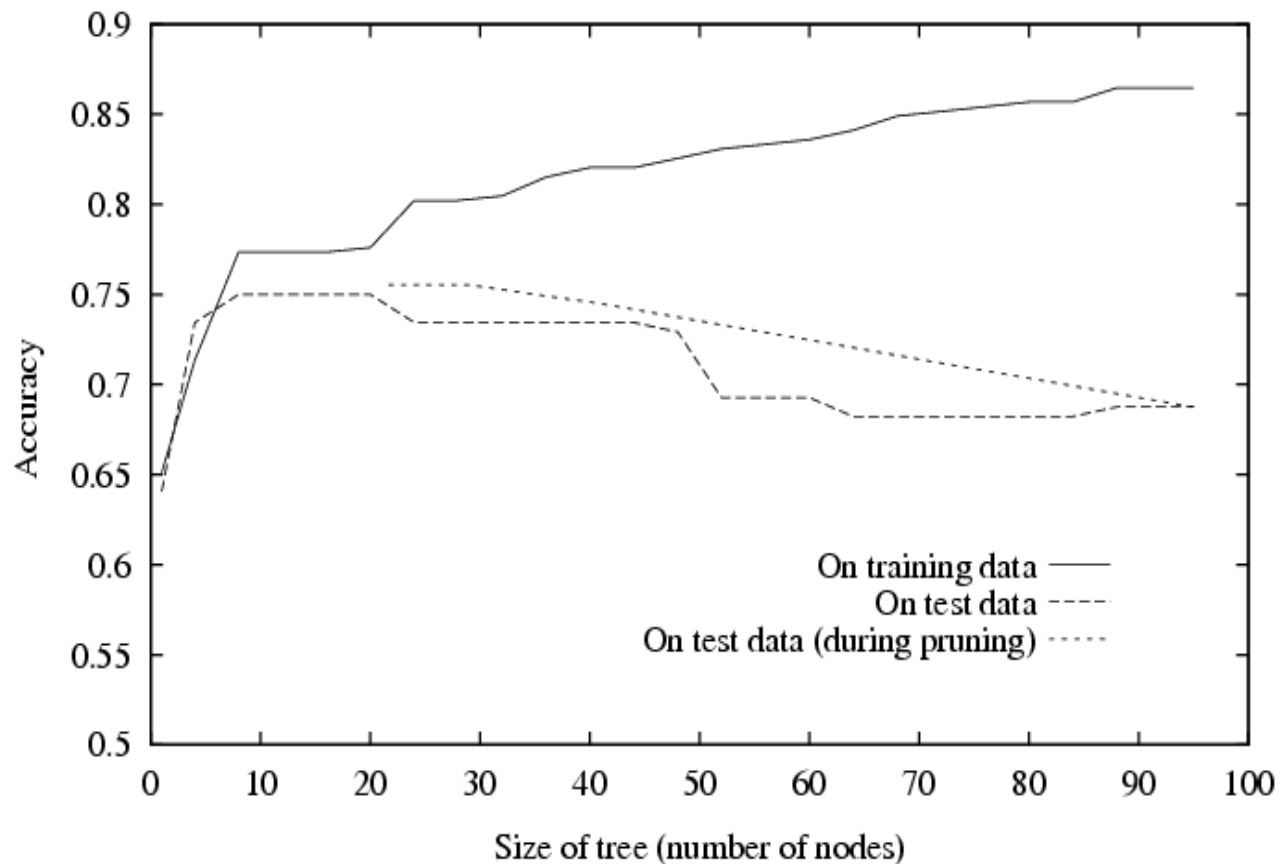
- Two types of pruning:
 - Pre-pruning (forward pruning)
 - Post-pruning (backward pruning)
- In pre-pruning, we decide during the building process when to stop adding features (possibly based on their information gain)
- However, this may be problematic – Why?
 - Sometimes features individually do not contribute much to a decision, but combined, they may have a significant impact

Decision Tree

- Post-pruning waits until the full decision tree has built and then prunes the attributes
- Subtree Replacement:
 - Entire subtree is replaced by a single leaf node



Effect of Reduced Error Pruning



Continuous Valued Attributes

Create a discrete attribute to test continuous

- Temperature = 24.5°C
- (Temperature > 20.0°C) = {true, false}

Where to set the threshold?

Temperature	15°C	18°C	19°C	22°C	24°C	27°C
PlayTennis	No	No	Yes	Yes	Yes	No

Unknown Attribute Values

What if some examples have missing values of A ?

Use training example anyway sort through tree

- If node n tests A , assign most common value of A among other examples sorted to node n .
- Assign most common value of A among other examples with same target value
- Assign probability p_i to each possible value v_i of A
 - Assign fraction p_i of example to each descendant in tree

Classify new examples in the same fashion

Summary for Decision Tree

- Types of decision trees
 - ID3 [Quinlan 1979, 1986]
 - CART: category and regression tree [Breiman et al., 1984]
 - C4.5 [Quinlan, 1993]
- How to choose attributes?
 - Gini Index (measurement of impurity)
 - Information Gain (entropy)
- Avoid overfitting
 - Tree pruning
- Continuous variables and unknown attribute values